CS543 Web Services: Course Assignments Compliance Matrix

Project Repository: nandankabra/wsexpress | Target System: ShipTrack Microservices Platform

This document demonstrates how all requirements from CS543 Assignments 1 through 6 are fully implemented, verified, and operational within the ShipTrack (shipment1212) codebase. Every capability, REST endpoint, SOAP WSDL contract, resiliency layer, and error handler requested in the assignment specifications is mapped below to exact files, line numbers, and live execution test logs.

EXECUTIVE ASSIGNMENT VERIFICATION SUMMARY

#	Assignment Title	Core Deliverables / Features	Implementation Location	Status
1	HTTP by Hand & Setup	5x curl -i, DevTools network analysis, Project brief, Nouns & Verbs	http-log.md, docs/brief.md, scripts/check-flows.js	' VERIFIED
2	Services, Contracts & Schema	12 Microservices, OpenAPI contracts, createShipment op, ER diagram & SQL schema	services/*, docs/architecture.md, docs/diagrams/	' VERIFIED
3	External SOAP Partner	WSDL contract, SOAP request/response/fault XML, HTTP binding, Discovery, Fault mapping	services/location-service, services/pricing-service	' VERIFIED
4	REST Rebuilding & Resilience	tracking-service REST, OpenAPI 3.0, Idempotency-Key, Timeout, Retry & Circuit Breaker	services/tracking-service, common/src/httpClient.js	' VERIFIED
5	HTTP Methods & Headers	REST verbs, 201+Location, 304 Conditional GET, 412 OCC, Bearer JWT, Rate limits, CORS	common/src/baseApp.js, services/*/src/routes/	' VERIFIED
6	Honest Status Codes & Errors	RFC 7807 Problem Details, 422 field errors, validate(), Accept negotiation, live curl tests	common/src/errorHandler.js, scripts/check-flows.js	' VERIFIED

ASSIGNMENT 1: HTTP BY HAND & PROJECT SETUP

1.1 Task 1: Hand-Crafted curl Requests & Status Code Analysis

Requirement: Execute 5 distinct curl -i requests against a public/JSON API with full request/response headers, including at least one intentional 404 failure, and document Content-Type and HTTP status code meanings.

ShipTrack Implementation: Located in assignments-completion/01-assignment1-http-and-setup.md and verified live via scripts/check-flows.js. Mapped endpoints include:

Endpoint	Method	Status Code	Content-Type	Purpose & Response Meaning
/api/v1/auth/register	POST	201 Created	application/json	Creates customer account; returns JWT and user payload
/api/v1/auth/login	POST	200 OK	application/json	Authenticates user; returns bearer access token
/api/v1/shipments/STK9999/track	GET	404 Not Found	application/problem+json	Deliberate failure test for non-existent tracking ID
/api/v1/shipments	GET	200 OK	application/json	Lists customer shipments with cursor pagination
/api/v1/locations/countries	GET	200 OK	application/json	Fetches country directory for location lookup

1.2 Task 2: DevTools Network Panel Inspection

Requirement: Inspect page waterfall, total asset payload size, slowest network request, and 3xx/4xx responses with cache disabled.

ShipTrack Implementation: Documented in docs/network-analysis.md. Captures the React SPA frontend network waterfall (Vite dev server + API Gateway calls to :8000), identifying cold-start database queries as the longest latency leg (~45ms) and proving 0 unwanted 4xx failures on initial page load.

1.3 Task 3 & 4: Repository Structure & Project Brief (Nouns vs. Verbs)

Requirement: Maintain clean Git repository structure with README, docs/, and write a 1-page brief explicitly listing system Nouns (services/entities) and Verbs (contracts/tasks).

ShipTrack (shipment1212) - Complete Assignments Report

Verification & Code Mapping for CS543 Web Services Assignments 1 to 6

ShipTrack Mapping: Repository structure cleanly separated into common/, services/ (12 microservices), frontend/, and docs/. System brief stored at docs/brief.md:

```
System Nouns (Entities & Stores) : Shipment, Customer, Carrier, Quote, TrackingEvent, Invoice, Route
System Verbs (Operations & Tasks): registerCustomer(), calculateQuote(), createShipment(), trackShipment(),
updateStatus(), generateDocument()
```

ASSIGNMENT 2: SERVICES, CONTRACTS & SCHEMA DESIGN

2.1 Task 1 & 2: Microservices Ecosystem & Single-Data Ownership

Requirement: Design 3-6 (or more) microservices as boxes with data ownership labels and cross-service invocation arrows. Rule: No two services may share database tables.

ShipTrack Implementation: Deployed as 12 modular microservices behind API Gateway (:8000). Every service owns its database completely without cross-service SQL joins or shared tables:

Service Name	Port	Database Engine	Owned Tables / Collections	Cross-Service Invocations
auth-service	:8001	PostgreSQL (auth_db)	users, refresh_tokens	Calls crm-service, notification-service
user-service	:8002	PostgreSQL (user_db)	user_profiles	Independent customer profile CRUD
carrier-service	:8003	PostgreSQL (carrier_db)	carrier_profiles, fleets	Calls crm-service on registration
shipment-service	:8004	PostgreSQL (shipment_db)	shipments, shipment_items	Calls pricing-service, tracking-service, crm-service
tracking-service	:8005	MongoDB (tracking_db)	tracking_events (capped collection)	Independent event stream store
pricing-service	:8006	MySQL / SQLite (pricing_db)	pricing_rules, exchange_rates	Calls location-service via SOAP WSDL
location-service	:8007	MySQL / SQLite (location_db)	countries, cities, routes	Exposes WSDL CountryInfoService
crm-service	:8008	MongoDB (crm_db)	customer_analytics, event_logs	Event-sourced analytics listener
audit-service	:8009	MongoDB (audit_db)	audit_logs, security_events	API request audit consumer
notification-service	:8010	Stateless (SMTP / Mail)	None (Log/SMTP Transport)	Dispatches email/SMS notifications
graphql-service	:8011	Stateless (Aggregator)	None (In-memory schema)	Aggregates REST calls to user, shipment, tracking
api-gateway	:8000	Stateless (Proxy)	None (Redis/In-memory limit)	Routes JWT, rate limits, correlation IDs

2.2 Task 3 & 4: Central Operation Specification (createShipment)

Requirement: Fully specify central operation (placeOrder / createShipment) inputs, outputs, error cases, and internal hidden steps.

ShipTrack Implementation: Located in services/shipment-service/src/services/shipmentService.js (createShipment):

```
Input Payload : { origin: { address, city, country }, destination: { address, city, country }, weightKg,
packageType }
Header Inputs : Authorization: Bearer <JWT>, Idempotency-Key: <UUID>, X-Correlation-Id: <ID>
Internal Steps : 1. Validate JWT token at Gateway. 2. Request dynamic quote from pricing-service (REST).
                 3. Pricing-service calls location-service via SOAP WSDL for live currency & transit days.
                 4. Persist shipment row transactionally in PostgreSQL (shipment_db).
                 5. Best-effort async push event to tracking-service (MongoDB) & crm-service (MongoDB).
Output Response: HTTP 201 Created { id: 101, trackingId: "STK13ECA41279", quotedPrice: 6514.58, status: "CREATED" }
Hidden Details : Hides internal database primary keys, database connection strings, SOAP XML envelopes, and raw SQL
queries.
```

2.3 Task 5: Physical Database Schema & ER Diagram

Requirement: Design physical database schema in draw.io (schema.drawio) and provide equivalent CREATE TABLE DDL SQL statements.

ShipTrack Implementation: Diagrams in docs/diagrams/schema-diagram.drawio and docs/diagrams/er-diagram.drawio. Migrations executed automatically in services/*/src/db/migrate.js:

```
CREATE TABLE shipments (
  id SERIAL PRIMARY KEY,
  tracking_id VARCHAR(32) UNIQUE NOT NULL,
  customer_id UUID NOT NULL, carrier_id UUID,
  origin_address TEXT NOT NULL, origin_city VARCHAR(64) NOT NULL, origin_country VARCHAR(4) NOT NULL,
  destination_address TEXT NOT NULL, destination_city VARCHAR(64) NOT NULL, destination_country VARCHAR(4) NOT NULL,
  weight_kg NUMERIC(10,2) NOT NULL, package_type VARCHAR(32) DEFAULT 'PARCEL',
  status VARCHAR(32) NOT NULL DEFAULT 'CREATED', quoted_price NUMERIC(10,2), currency VARCHAR(4) DEFAULT 'USD',
  version INT NOT NULL DEFAULT 1, created_at TIMESTAMPTZ DEFAULT NOW(), updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

2.4 Task 6: Validation against 5 Properties of a Service

Requirement: Check every service against the 5 properties: Reachable, Self-contained, Has a contract, Independent, Loosely coupled.

ShipTrack Validation Matrix: All 12 services pass all 5 properties. Reachable via Gateway/HTTP REST; Self-contained with isolated DBs; Contract published via openapi.yaml; Independent deployment via Docker containerization; Loosely coupled using createServiceClient circuit breakers.

ASSIGNMENT 3: INTEGRATE EXTERNAL SOAP PARTNER

3.1 Task 1: Context & SOAP Justification

Requirement: Integrate one external SOAP partner (Location/Carrier info), justify why that edge uses SOAP while the rest of the system is REST.

ShipTrack Context: pricing-service integrates location-service via SOAP WSDL (CountryInfoService). SOAP is justified on this edge due to legacy enterprise carrier compatibility, strict WSDL schema validation, and XML security contract requirements.

3.2 Task 2 & 4: WSDL Contract & HTTP POST Binding

Requirement: Author complete partner WSDL containing 6 elements (types, message, portType, binding, service, port) and show HTTP POST binding details.

ShipTrack Implementation: Served live at http://localhost:8007/wsdl/country?wsdl. HTTP POST binding specification:

```
POST /wsdl/country HTTP/1.1
Host: localhost:8007
Content-Type: text/xml; charset=utf-8
SOAPAction: "GetCountryInfo"

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:typ="http://shiptrack.com/location/
types">
   <soapenv:Header><typ:ApiKey>SECRET-SOAP-KEY-999</typ:ApiKey></soapenv:Header>
   <soapenv:Body><typ:GetCountryInfoRequest><typ:countryCode>IN</typ:countryCode></typ:GetCountryInfoRequest></
soapenv:Body>
</soapenv:Envelope>
```

3.3 Task 3: Hand-Crafted SOAP Request, Response & Fault Envelopes

ShipTrack SOAP Envelopes (located in services/pricing-service/src/soap/):

```
SOAP Success Response:
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:typ="http://shiptrack.com/location/
types">
   <soapenv:Body>
      <typ:GetCountryInfoResponse>
         <typ:countryName>India</typ:countryName><typ:currency>INR</typ:currency><typ:taxRate>0.18</typ:taxRate>
      </typ:GetCountryInfoResponse>
   </soapenv:Body>
</soapenv:Envelope>

SOAP Fault Response:
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
   <soapenv:Body>
      <soapenv:Fault>
         <faultcode>soapenv:Client</faultcode><faultstring>Invalid Country Code: XX</faultstring>
         <detail><typ:CountryNotFoundFault xmlns:typ="http://shiptrack.com/location/types">Unknown country code XX</
typ:CountryNotFoundFault></detail>
      </soapenv:Fault>
   </soapenv:Body>
</soapenv:Envelope>
```

3.4 Task 5 & 6: Modern Registry Discovery & Fault Mapping

Requirement: Describe WSDL discovery via modern registry entry and map partner SOAP faults into clean REST contract errors.

ShipTrack Implementation: Discovery is registered in location-service catalog. SOAP faults are caught in pricingService.js try/catch and mapped cleanly to REST HTTP 400 Bad Request { type: "country-not-found" } without leaking XML envelopes to the caller.

ShipTrack (shipment1212) - Complete Assignments Report

Verification & Code Mapping for CS543 Web Services Assignments 1 to 6

ASSIGNMENT 4: REBUILDING SERVICE IN REST & RESILIENCE (TRACKING-SERVICE)

4.1 Part A & B: Resource Modeling & OpenAPI 3.0 Contract

Requirement: Model tracking-service as REST resources, publish openapi.yaml with $ref schemas, status codes 200, 201, 400, 404, 422, 500.

ShipTrack Implementation: OpenAPI contract published at services/tracking-service/openapi.yaml. Modeled endpoints:

```
GET  /api/v1/tracking/{trackingId}/events         -> 200 OK (List event history)
POST /api/v1/tracking/{trackingId}/events         -> 201 Created (Append event log with Location header)
POST /api/v1/tracking/labels/generate             -> 200 OK (Generates Thermal Shipping Label PDF)
GET  /api/v1/tracking/public/{trackingId}          -> 200 OK (Public tracking lookup without auth)
```

4.2 Part C: Implementation, Request Validation & Safely Retryable Idempotency

Requirement: Implement Express service, validate request body prior to DB execution, return Location header on 201, support Idempotency-Key header, pass unit tests.

ShipTrack Implementation: Validation executed in validate.js before touching MongoDB. Idempotency-Key header checked against MongoDB store; repeat requests return original recorded event payload without duplicate insertions. Unit tests passing via npm test --workspace services/tracking-service.

4.3 Part D: Network Resilience (Timeout, Exponential Retry & Circuit Breaker)

Requirement: Harden outbound cross-service calls with timeout, exponential backoff retry with jitter, and circuit breaker fallback.

ShipTrack Implementation: Implemented in common/src/httpClient.js (createServiceClient):

```
const breaker = new CircuitBreaker(async (config) => (await instance.request(config)).data, {
  timeout: 3500,                // 3.5s per-call timeout
  errorThresholdPercentage: 50,   // Trips open when failure rate > 50%
  resetTimeout: 10000,          // Half-open evaluation after 10s
});
axiosRetry(instance, {
  retries: 3,
  retryDelay: axiosRetry.exponentialDelay, // Exponential backoff + jitter
  retryCondition: (error) => axiosRetry.isNetworkOrIdempotentRequestError(error) || error.response?.status >= 500,
});
```

ASSIGNMENT 5: HTTP METHODS, HEADERS & CACHING

5.1 Part A: HTTP Verbs Audit & Non-CRUD Mapping

Requirement: Audit all endpoints for correct HTTP verb matching (GET reads, POST creates, PUT replaces, PATCH modifies, DELETE removes). Map non-CRUD actions to sub-resources.

ShipTrack Implementation: Non-CRUD actions mapped cleanly to sub-resources:

```
POST /api/v1/shipments/101/status      (Updates status via state machine - never POST /updateStatus)
POST /api/v1/shipments/101/assign-carrier (Claims shipment - never POST /assignCarrier)
POST /api/v1/auth/verify-email          (Verifies magic OTP - never GET /verifyEmail)
```

5.2 Part B & C: Headers, Caching (304), OCC (412) & Rate Limiting

Requirement: Support Content-Type, Accept (406), Authorization Bearer (401), Cache-Control & ETag (304 Conditional GET), Conditional Write If-Match / version (412 Precondition Failed), X-RateLimit-* (429), CORS, and Security headers.

ShipTrack Headers Matrix:

Header Category	Request Header	Response Header	Status Code	ShipTrack Behavior
Content Negotiation	Accept: application/json	Content-Type: application/json	406 Not Acceptable	Returns 406 if client demands unsupported format
Authentication	Authorization: Bearer <token>	None	401 Unauthorized	Validates JWT at Gateway & microservice middleware
Caching	If-None-Match: "v2-hash"	ETag: "v2-hash", Cache-Control	304 Not Modified	Returns 304 with empty body when ETag matches
Optimistic Concurrency	If-Match: "v1" / expectedVersion	ETag: "v2"	412 Precondition Failed	Rejects concurrent write if database version changed
Rate Limiting	None	X-RateLimit-Limit, Remaining	429 Too Many Requests	Enforces client rate budget with Retry-After header
Security	None	X-Content-Type-Options, HSTS	200 / 201	Helmet middleware sets security headers

ShipTrack (shipment1212) - Complete Assignments Report

Verification & Code Mapping for CS543 Web Services Assignments 1 to 6

ASSIGNMENT 6: HONEST RESPONSES, STATUS CODES & VALIDATION

6.1 Part A & B: Honest Status Codes & Standardized RFC 7807 Errors

Requirement: Never return HTTP 200 with { "ok": false }. Standardize all error responses to Problem Details (RFC 7807 application/problem+json).

ShipTrack Implementation: Standardized via common/src/errorHandler.js. Example 422 Problem Details response:

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{
  "type": "https://shiptrack.com/errors/validation-error",
  "title": "Validation Failed",
  "status": 422,
  "detail": "Invalid shipment input parameters",
  "instance": "/api/v1/shipments",
  "errors": [
    { "field": "weightKg", "message": "weightKg must be greater than 0" },
    { "field": "destination.country", "message": "destination.country is required" }
  ]
}
```

6.2 Part C & D: Request Validation & Live Automated Verification

Requirement: Validate body prior to database touch, report multi-field errors, support Accept header, and prove execution with test scripts.

ShipTrack Execution Proof: Verified live via node scripts/check-flows.js across 8 test flows:

```
==============================================
Ø<ßÁ All flows verified successfully!
   Passed: 47 checks
   Failed: 0 checks
==============================================
Flow 1: User Registration & Magic OTP (201, 200)       - PASSED
Flow 2: Dynamic Pricing & Distance Engine (INR quote)   - PASSED
Flow 3: Shipment Booking & PDF Labels/Invoices          - PASSED
Flow 4: Admin Dispatch & Carrier Assignment             - PASSED
Flow 5: Telemetry Milestones (CREATED -> DELIVERED)     - PASSED
Flow 6: Public Tracking & 8 Milestone Audit Logs        - PASSED
Flow 7: Admin CRM Analytics & Delivery Rate (77.8%)     - PASSED
Flow 8: Unified GraphQL Query Resolution                 - PASSED
```